OPERATING SYSTEMS PROJECT REVIEW | CO: 1 TO CO: 5 COMPLETE COVERAGE | C99 / POSIX

Mini-Container Runtime

A Lightweight Linux Container Virtualization Engine Built from Scratch in C99
Utilizing Linux Namespaces, Unified Cgroups v2, and VFS Sandboxing

TEAM MEMBER 1

Tharakesha M

Reg No: 24BCE2538

B.Tech Computer Science & Engineering
Core Developer (Namespaces & Cgroups)

TEAM MEMBER 2

Mathu Metha S

Reg No: 24BCE2805

B.Tech Computer Science & Engineering
Core Developer (Concurrency & Testing)

GitHub: **github.com/THARAKESHA/mini-container**

Introduction: The Rise of Containerization

BACKGROUND & CONTEXT

The Evolution of Cloud Compute

- **Monolithic Deployments:** Traditional applications ran on bare metal, leading to dependency conflicts.
- **Hardware Virtualization:** Hypervisors (VMware, KVM) introduced isolation, but at high hardware overhead.
- **The Container Revolution:** Docker and Kubernetes replaced heavy VMs with lightweight process sandboxes.
- **Modern Enterprise Standard:** Over 90% of enterprise cloud workloads run inside container runtimes (runc, containerd).
- **The Engineering Question:** How does the OS kernel achieve isolation without hardware emulation?

Project Purpose: Demystifying Containers

- **The "Black Box" Problem:** Developers treat Docker as magic without understanding low-level POSIX system calls.
- **Zero-Dependency Implementation:** We build an entire container engine from scratch in pure C99.
- **Native Linux Kernel Primitives:** Direct interaction with Namespaces, Cgroups v2, and VFS.
- **Academic Rigor:** Explicitly models all five Course Outcomes (CO: 1 to CO: 5) of the Operating Systems syllabus.
- **Systems Engineering:** Proves hands-on systems programming skills valued in backend and infrastructure roles.

Problem Statement: Containers vs. Virtual Machines

COMPARATIVE ANALYSIS

Traditional Virtual Machines (Hypervisors)

- Virtualizes the physical **hardware layer** using a hypervisor (Type-1/2).
- Requires a dedicated, redundant guest OS kernel per instance.
- High memory footprint (1 - 4 GB) and slow boot latency (30 - 60s).
- Incurs hardware emulation translation penalties and CPU traps.
- Rigid, coarse-grained hardware resource partitioning.

OS-Level Containers (mini-container in C)

- Shares the single underlying host Linux kernel with zero emulation.
- Containers are regular OS processes wrapped in kernel-enforced limits.
- Sub-second boot times (10-20ms) and minimal RAM overhead (< 5MB).
- Near-native bare-metal execution performance with hardware quotas.
- Fine-grained dynamic control via modern Linux Cgroups v2.

EFFICIENCY COMPARISON BENCHMARK:

Boot Latency: **VM (45,000 ms)** vs. **Container (15 ms)** | Memory Overhead: **VM (~1500 MB)** vs. **Container (< 5 MB)** | CPU Overhead: **~0.1%**

Literature Survey: Container Runtimes Landscape

Technology	Isolation Mechanism	Language & Dependencies	Primary Use Case
Docker (runc)	Namespaces + Cgroups v1/v2	Go (Heavy daemon, containerd)	Enterprise Cloud Application Containers
LXC / LXD	Linux Namespaces + AppArmor	C / Go (System containers)	Full OS Virtualization Containers
QEMU / KVM	Hardware Hypervisor (VT-x)	C / Assembly (Guest kernel required)	Full Hardware Virtualization
mini-container (Ours)	Namespaces + Unified Cgroups v2	Pure C99 (Zero third-party libraries)	Educational Systems Research & Verification

Syllabus Alignment: Complete CO 1 to CO 5 Coverage

COURSE OUTCOMES MATRIX

Course Outcome (CO)	Syllabus Topics Covered	Implementation Mechanism in C
CO 1: OS Principles & Processes	System calls, User/Kernel modes, PCB, Process creation	`clone()` with isolated stack, `waitpid()`, custom PCB handling
CO 2: CPU Scheduling	CFS quotas, Preemption, Proportional Fair Share	cgroups `cpu.max` (40% quota) & `cpu.weight` (4:1 contest)
CO 3: Concurrency & IPC	Synchronization primitives, Semaphores, Shared Memory	POSIX `sem_t` (mutex, empty, full) & `mmap` ring buffer
CO 4: Memory Management	Virtual memory, Demand paging, Page faults, OOM	cgroups `memory.max`, `pgfault` tracking (7,705 faults), `SIGKILL`
CO 5: Virtualization & Filesystems	Container virtualization, Mount propagation, VFS	Namespaces (`PID`, `UTS`, `Mount`, `IPC`), `chroot()`, private `/proc`

System Architecture: Layered Kernel Abstractions

SYSTEM ARCHITECTURE

Host vs. Container Abstraction Layers

- **User Space Layer (CLI):** `minicontainer` CLI parses commands and parameters (--memory, --cpu, --pids, --hostname, --rootfs).
- **Host Orchestrator:** Allocates 1MB stack, sets up sync_pipe, and coordinates child lifecycles with waitpid().
- **Kernel Subsystem Boundary:** `clone()` creates isolated namespaces; `/sys/fs/cgroup/` binds hardware quotas.
- **Container Sandbox:** Child executes as PID 1, completely isolated from host visibility.
- **Virtual File System:** Isolated rootfs (Alpine Linux) with private `/proc` pseudo-filesystem.

Key Kernel Interfaces Utilized

- `sched.h (clone)`: Bitwise namespace flags (`CLONE_NEWPID`, `CLONE_NEWUTS`, `CLONE_NEWNS`, `CLONE_NEWIPC`).
- `sys/mount.h`: VFS mount isolation and `MS_PRIVATE` propagation.
- `unistd.h (chroot)`: Root directory confinement.
- `/sys/fs/cgroup`: Linux unified cgroup v2 controller interface.
- `semaphore.h & sys/mman.h`: Concurrency synchronization and anonymous shared memory.

Execution Lifecycle: Coordinated Process Pipeline

PROCESS LIFECYCLE FLOW

Coordinated Execution Sequence

```
1. CLI Parsing:
   └─► sudo minicontainer run --memory 30M --cpu 40 /bin/sh

2. Host Parent Process:
   ├─► Allocates 1MB stack & creates UNIX sync pipe (barrier)
   ├─► clone(CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS | ...)
   ├─► Creates /sys/fs/cgroup/minicontainer-[PID]
   ├─► Applies memory.max, cpu.max, pids.max
   ├─► Attaches child PID to cgroup.procs
   ├─► Signals child via pipe (unblocks execution)
   └─► Calls waitpid(), reads memory.stat, cleans cgroup

3. Container Child Process:
   ├─► Blocks on pipe until cgroups are attached
   ├─► sethostname("container-box")
   ├─► chroot() to rootfs & mounts private /proc
   └─► execvp(argv[0]) as PID 1
```

Process State Machine & Safety

- **State 1 (CREATED):** Child spawned via `clone()` but blocked on pipe.
- **State 2 (BOUND):** Parent completes cgroup controller configuration.
- **State 3 (JAILED):** Child unblocked; executes `chroot()` & private mounts.
- **State 4 (RUNNING):** `execvp()` overwrites address space (becomes PID 1).
- **State 5 (TERMINATED):** Normal exit or OOM Killer `SIGKILL`.
- **State 6 (RECLAIMED):** Parent `waitpid()` catches status, cleans cgroups.

Core Subsystem 1: Linux Namespaces (Visibility)

KERNEL NAMESPACES

PID Namespace (CLONE_NEWPID)

- Virtualizes the process ID hierarchy.
- Container payload becomes **PID 1** inside the container.
- Host processes (47 active) are completely hidden.
- Signals (`SIGKILL` / `SIGSTOP`) from container cannot reach host.

UTS Namespace (CLONE_NEWUTS)

- Isolates nodename (hostname) and NIS domain name.
- Allows container to execute `sethostname('isolated-box')`.
- Host OS hostname remains untouched as `Tharakesha`.
- Prevents multi-tenant hostname collisions.

Mount Namespace (CLONE_NEWNS)

- Decouples the Virtual File System (VFS) mount table.
- Enforces `MS_PRIVATE` mount propagation.
- Mounting private `/proc` does not alter host `/proc`.

IPC Namespace (CLONE_NEWIPC)

- Isolates System V IPC and POSIX message queues.
- Shared memory segments and semaphores are private.
- Prevents cross-container data snooping.

Core Subsystem 2: Unified Cgroups v2 (Resource Envelopes)

RESOURCE CONTROL

Memory Ceiling (memory.max)

- Defines hard byte threshold on physical DRAM (e.g., 30MB).
- Sets `memory.swap.max = 0` to prevent paging to disk.
- Forces kernel Out-Of-Memory (OOM) Killer intervention.
- Protects host from memory exhaustion.

CPU Bandwidth Quota (cpu.max)

- CFS Bandwidth Control: `quota period` (e.g. `40000 100000`).
- Limits container compute to deterministic 40% bandwidth.
- Scheduler removes task from runqueue when quota spent.
- Prevents runaway threads from starving host CPU.

Proportional Share (cpu.weight)

- Weighted fair queuing for concurrent workloads (1 - 10000).
- Weights 800 vs 200 allocate 80% and 20% on Core 0.
- Empirically validated exact 4.02 : 1 execution ratio.

Process Count Cap (pids.max)

- Imposes ceiling on total concurrent tasks.
- Setting `pids.max = 10` blocks fork bomb DoS attacks.
- 10th `fork()` fails immediately with `errno 11 (EAGAIN)`.

Core Subsystem 3: Filesystem Sandboxing & VFS

FILESYSTEM ISOLATION

Root Filesystem Jailing (chroot)

- Confines the process to a self-contained directory tree (`./rootfs`).
- Leverages official Alpine Linux minirootfs (~3.5MB compressed).
- Provides full POSIX userland utilities (`/bin/sh`, `/bin/ls`, `/bin/ps`).
- `chroot(rootfs)` resets apparent root directory (/) to container directory.
- Prevents directory traversal above the container boundary.
- Containerized programs cannot access host files or Windows drives.

Private /proc VFS Mount Isolation

- The Linux `/proc` directory is a kernel-generated pseudo-filesystem containing active process status.
- Without private `/proc`, running `ps aux` inside a container would expose all host processes!
- Our runtime enforces `mount(NULL, '/', NULL, MS_REC | MS_PRIVATE, NULL)` to isolate mount propagation.
- Mounts fresh procfs inside container: `mount('proc', '/proc', 'proc', 0, NULL)`.
- Result: `ps aux` displays only `PID 1 (/bin/sh)` and `PID 2 (ps aux)`.

Classical Concurrency: Producer-Consumer with Semaphores (CO: 3)

CONCURRENCY PRIMITIVES

Bounded-Buffer Synchronization Model

- **Classical Bounded-Buffer Concurrency Problem** implemented inside the container sandbox.
- **Binary Semaphore (mutex):** Initialized to 1; enforces mutual exclusion for the critical section.
- **Counting Semaphore (empty_slots):** Initialized to buffer capacity (N = 5).
- **Counting Semaphore (full_slots):** Initialized to 0 items available.
- **Shared Memory (mmap):** Allocated via `MAP_SHARED | MAP_ANONYMOUS` across processes.
- **Zero Race Conditions:** 10 items produced and consumed with zero race conditions or buffer corruption.

Parent-Child IPC Pipe Synchronization Barrier

- **The Race Condition Hazard:** Cloned child process could execute payload before parent attaches cgroups limits.
- **The POSIX Pipe Solution:** Parent and child share a unidirectional `sync_pipe[2]`.
- **Child Execution:** Calls `read(sync_pipe[0])` and immediately **BLOCKS**.
- **Parent Execution:** Completes cgroup setup, then writes single trigger byte `'G'`.
- **Unblocking:** Child receives byte, closes pipe, and proceeds safely to `execvp()`.
- Ensures deterministic, race-free resource limits from microsecond zero.

Virtual Memory: Demand Paging & OOM Killer (CO: 4)

MEMORY MANAGEMENT

Demand Paging & Page Fault Accounting

- **Optimistic Memory Allocation:** `malloc()` reserves virtual address space without allocating physical DRAM frames.
- **Demand Paging Traps:** When `memset()` accesses pages, the CPU MMU triggers Page Faults to map physical frames.
- **Kernel Metric Tracking:** Reads `/sys/fs/cgroup/.../memory.stat` upon container conclusion.
- **Minor Page Faults Recorded:** Exactly **7,705 Minor Page Faults** logged during 30MB allocation.
- **Major Page Faults:** Exactly 1; proves zero disk thrashing occurred.
- **Empirical Proof:** Directly proves demand paging mechanisms in the operating system kernel.

Kernel Out-Of-Memory (OOM) Killer Intervention

- **Resource Constraint:** Container configured with strict 30MB memory ceiling (`memory.max = 31457280`).
- **Swap Disabled:** `memory.swap.max = 0` forces immediate physical memory enforcement.
- **Stress Workload:** `test_memory` attempts to allocate 60MB in 5MB steps.
- **Kernel Intervention:** At 30MB, the Linux kernel Out-Of-Memory Killer steps in.
- **Signal Dispatch:** Sends uncatchable **SIGKILL (signal 9)** to terminate the container.
- **Parent Detection:** `WIFSIGNALED(status)` detects SIGKILL and reports clean diagnostic.

CPU Scheduling: CFS Quotas & Fair-Share Contest (CO: 2)

CPU SCHEDULING

CFS Bandwidth Quota Throttling (cpu.max)

- **CFS Scheduler Model:** Linux allocates CPU time over 100ms periods.
- **Quota Applied:** `--cpu 40` configures quota = 40,000 us per 100,000 us period.
- **Workload:** `test_cpu` executes an intensive infinite compute loop for 3 wall-clock seconds.
- **Observed CPU Time:** Clamped at 1.12 seconds CPU time (**37.4%** effective CPU).
- **Preemption Guarantee:** Completely Fair Scheduler suspends task once quota is spent, protecting host cores.

Proportional Share Contest: 800 vs. 200 (cpu.weight)

- **Contest Design:** Two concurrent containers pinned to CPU Core 0 via `taskset -c 0`.
- **Priority Configuration:** Container A (weight 800 ~ 80%) vs. Container B (weight 200 ~ 20%).
- **Observed Execution Time:** Container A: **2.78s** CPU Time | Container B: **0.69s** CPU Time.
- **Empirical Ratio:** 2.78 / 0.69 = **4.02 : 1** execution ratio.
- **Validation:** Matches the theoretical **4 : 1** weight ratio with **99.5% accuracy**!

Security & DoS Defense: Fork Bomb Mitigation (CO: 1 & 3)

SECURITY CONTROLS

The Fork Bomb Vulnerability

- **Mechanism of Attack:** A malicious process recursively calls `fork()` in an infinite loop.
- **Resource Depletion:** Rapidly exhausts the operating system's Process Control Block (PCB) table.
- **System Consequence:** Unconfined machines freeze entirely; administrators cannot launch terminals or kill tasks.
- **Relevance:** Demonstrates operating system deadlock, starvation, and process resource exhaustion.

Cgroups v2 Task Limiting (pids.max)

- **Hardware Defense:** Setting `pids.max = 10` caps total concurrent threads/processes.
- **Laboratory Stress Test:** `test_forkbomb` attempts to spawn 50 processes.
- **Interception:** Children 1 through 9 spawn successfully (10 total tasks including parent).
- **Rejection:** 10th `fork()` blocked with **errno 11 (EAGAIN: Resource temporarily unavailable)**.
- **Host Stability:** Host machine remained 100% responsive throughout the attack.

Empirical Laboratory Benchmarks & Observed Results OBSERVED METRICS

1. Process Virtualization (CO: 1 & 5)

• Container Process reports **PID = 1**; Parent PID = 0.
• Host operating system processes (47 active) are completely hidden.
• Hostname virtualized to `isolated-box` without altering host.

2. Memory Ceiling & Demand Paging (CO: 4)

• Workload allocated 30MB of 60MB; terminated by **SIGKILL (signal 9)**.
• Kernel logged **7,705 Minor Page Faults**, validating Demand Paging.
• 1 Major Page Fault; Swap disabled to force immediate OOM protection.

3. Proportional Fair-Share CPU Contest (CO: 2)

• Competing containers on CPU Core 0 with CFS weights 800 vs 200.
• Container A CPU Time: **2.78s** | Container B CPU Time: **0.69s**.
• Exact **4.02 : 1** execution ratio (matches theoretical 4:1 with 99.5% accuracy!).

4. Fork Bomb & PCB Table Defense (CO: 1 & 3)

• Stress test attempted to spawn 50 concurrent processes.
• Cgroups `pids.max = 10` capped execution; Children 1 - 9 spawned.
• 10th fork() blocked with **errno 11 (EAGAIN)**; host OS remained responsive.

EVALUATION & LIVE DEMONSTRATION

Interactive Project Demonstration

Complete C99 Runtime Covering CO: 1 to CO: 5 with Master Interactive Menu and Real-Time Kernel Metrics

```
$ sudo bash interactive_menu.sh
```

(Or double-click **Run-MiniContainer.bat** directly on your Windows Desktop)

Project Team: Tharakesha M (24BCE2538) & Mathu Metha S (24BCE2805)

GitHub: https://github.com/THARAKESHA/mini-container